

1.206.Reverse Linked List

```
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        prev = None
        while head:
            head.next, prev, head = prev, head, head.next
        return prev
```

2. 203. Remove Linked List Elements

```
class Solution:
    def removeElements(self, head: Optional[ListNode], val: int) -> Optional[ListNode]:
        dummy = ListNode(0)
        dummy.next = head
        curr = dummy
        while curr and curr.next:
            if curr.next.val == val:
                curr.next = curr.next.next
            else:
                curr = curr.next
        return dummy.next
```

3. 876. Middle of the Linked List

```
class Solution:
    def middleNode(self, head: Optional[ListNode]) -> Optional[ListNode]:
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        return slow
```

4. 237. Delete Node in a Linked List

```
class Solution:
    def deleteNode(self, node: Optional[ListNode]) -> None:
```

```
node.val = node.next.val
node.next = node.next.next
```

5. 1290. Convert Binary Number in a Linked List to Integer

```
class Solution:
    def getDecimalValue(self, head: Optional[ListNode]) -> int:
        num = 0
        while head:
            num = num * 2 + head.val
            head = head.next
        return num
```

6. 83. Remove Duplicates from Sorted List

```
class Solution:
    def deleteDuplicates(self, head: Optional[ListNode]) -> Optional[ListNode]:
        curr = head
        while curr and curr.next:
            if curr.val == curr.next.val:
                curr.next = curr.next.next
            else:
                curr = curr.next
        return head
```

7. 21. Merge Two Sorted Lists

```
class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode()
        curr = dummy
        while list1 and list2:
            if list1.val < list2.val:
                curr.next = list1
                list1 = list1.next
            else:
                curr.next = list2
                list2 = list2.next
            curr = curr.next
```

```
curr.next = list1 or list2
return dummy.next
```

8. 141. Linked List Cycle

```
class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                return True
        return False
```

9. 234. Palindrome Linked List

```
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        vals = []
        while head:
            vals.append(head.val)
            head = head.next
        return vals == vals[::-1]
```

10. 160. Intersection of Two Linked Lists

```
class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) -> Optional[ListNode]:
        a, b = headA, headB
        while a != b:
            a = a.next if a else headB
            b = b.next if b else headA
        return a
```

11. 708. Insert into a Sorted Circular Linked List

```

class Solution:
    def insert(self, head: 'Optional[Node]', insertVal: int) -> 'Node':
        newNode = Node(insertVal)
        if not head:
            newNode.next = newNode
            return newNode
        curr = head
        while True:
            if curr.val <= insertVal <= curr.next.val or \
               (curr.val > curr.next.val and (insertVal >= curr.val or insertVal <= curr
               curr.next == head:
                newNode.next = curr.next
                curr.next = newNode
                break
            curr = curr.next
        return head

```

12. 707. Design Linked List

```

class Node:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class MyLinkedList:

    def __init__(self):
        self.head = None

    def get(self, index: int) -> int:
        curr = self.head
        for _ in range(index):
            if not curr: return -1
            curr = curr.next
        return curr.val if curr else -1

    def addAtHead(self, val: int) -> None:
        self.head = Node(val, self.head)

    def addAtTail(self, val: int) -> None:
        if not self.head:
            self.head = Node(val)
            return
        curr = self.head
        while curr.next:

```

```
        curr = curr.next
curr.next = Node(val)

def addAtIndex(self, index: int, val: int) -> None:
    if index == 0:
        self.addAtHead(val)
        return
    curr = self.head
    for _ in range(index - 1):
        if not curr: return
        curr = curr.next
    if curr:
        curr.next = Node(val, curr.next)

def deleteAtIndex(self, index: int) -> None:
    if index == 0:
        if self.head:
            self.head = self.head.next
        return
    curr = self.head
    for _ in range(index - 1):
        if not curr: return
        curr = curr.next
    if curr and curr.next:
        curr.next = curr.next.next
```